

LAB PROGRAM – 01

Build a User List API using Flask:

- a. Create a Flask server with endpoint /users returning a static list of 5 users (JSON).
- b. Add endpoint /users/ to return details of a specific user.

Code: Create a file called as lab1.py

```
from flask import Flask, jsonify
app = Flask(__name__)
users = [
    {
        "id": 1,
        "name": "Modi",
        "email": "Modi@example.com",
        "age": 25
    },
    {
        "id": 2,
        "name": "Nirmala",
        "email": "nirmala@example.com",
        "age": 25
    },
    {
        "id": 3,
        "name": "Seetha",
        "email": "seetha@example.com",
        "age": 28
    },
    {
        "id": 4,
        "name": "David",
        "email": "david@example.com",
```

```

        "age": 24
    },
    {
        "id": 5,
        "name": "Geetha",
        "email": "geetha@example.com",
        "age": 21
    }
]

# Endpoint to return all users
@app.route('/users', methods=['GET'])
def get_users():
    return jsonify({
        "success": True,
        "message": "Users retrieved successfully.",
        "count": len(users),
        "data": users
    }), 200

# Endpoint to return a specific user by ID
@app.route('/users/<int:id>', methods=['GET'])
def get_user(id):
    for user in users:
        if user["id"] == id:
            return jsonify({
                "success": True,
                "message": "User retrieved successfully.",
                "data": user
            }), 200
    return jsonify({"message": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Steps to execute:

Activate Environment : .venv\Scripts\activate

Run the server : python lab1.py

```
PS C:\Users\lavus\Desktop\SVRFGC\BD\Lab> .venv\Scripts\activate
(.venv) PS C:\Users\lavus\Desktop\SVRFGC\BD\Lab> python lab1.py
* Serving Flask app 'lab1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-284-215
```

Test the API

a. Get all Users

Method: GET

URL: <http://127.0.0.1:5000/users>

Response:

```
{
  "count": 5,
  "data": [
    {
      "age": 25,
      "email": "Modi@example.com",
      "id": 1,
      "name": "Modi"
    },
    {
      "age": 25,
      "email": "nirmala@example.com",
      "id": 2,
      "name": "Nirmala"
    },
  ],
}
```

```
{
  "age": 28,
  "email": "seetha@example.com",
  "id": 3,
  "name": "Seetha"
},
{
  "age": 24,
  "email": "david@example.com",
  "id": 4,
  "name": "David"
},
{
  "age": 21,
  "email": "geetha@example.com",
  "id": 5,
  "name": "Geetha"
}
],
"message": "Users retrieved successfully.",
"success": true
}
```

Status Code: 200 OK

b. Get User by ID

Method: GET

URL: <http://127.0.0.1:5000/users/1>

Response:

```
{
  "data": {
    "age": 25,
    "email": "Modi@example.com",
    "id": 1,
```

```
"name": "Modi"
},
"message": "User retrieved successfully.",
"success": true
}
```

Status Code: 200 OK

c. Invalid User

Request: <http://127.0.0.1:5000/users/6>

Response:

```
{
  "message": "User not found"
}
```

Status Code: 404 NOT FOUND

LAB PROGRAM – 02

Create a Student Registration API:

- a. Build a Flask API with endpoint /register to accept name and email using POST.
- b. Validate email format and return success or error message.

Code: Create a file called as lab2.py

```
from flask import Flask, request, jsonify
import re

app = Flask(__name__)

# Email validation function
def is_valid_email(email):
    pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
    return re.match(pattern, email)

# Student Registration API
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    name = data.get("name")
    email = data.get("email")

    if not name or not email:
        return jsonify({
            "status": "Error",
            "message": "Name and Email are required."
        }), 400

    # Validate email
    if not is_valid_email(email):
```

```

        "status": "Error",
        "message": "Invalid email format."
    }), 400

# Success response
return jsonify({
    "status": "Success",
    "message": "Student registered successfully.",
    "student": {
        "name": name,
        "email": email
    }
}), 201

if __name__ == '__main__':
    app.run(debug=True)

```

Steps to execute:

Activate Environment : .venv\Scripts\activate

Run the server : python lab2.py

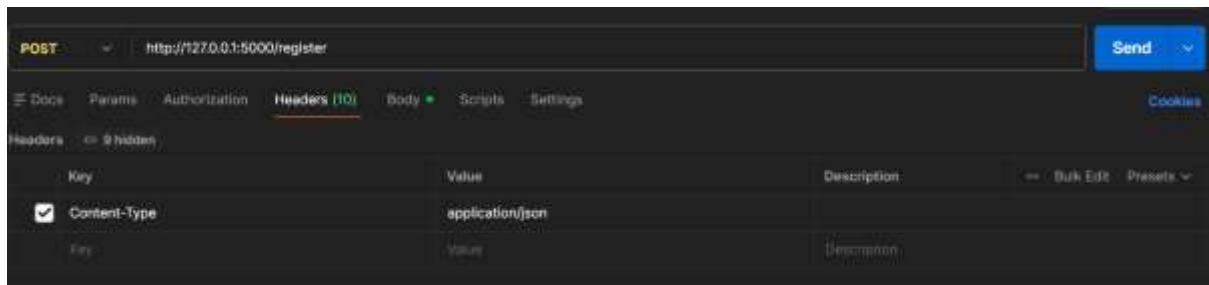
```

(.venv) PS C:\Users\lavus\Desktop\SVRFGC\BD\Lab> python lab2.py
* Serving Flask app 'lab2'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-284-215

```

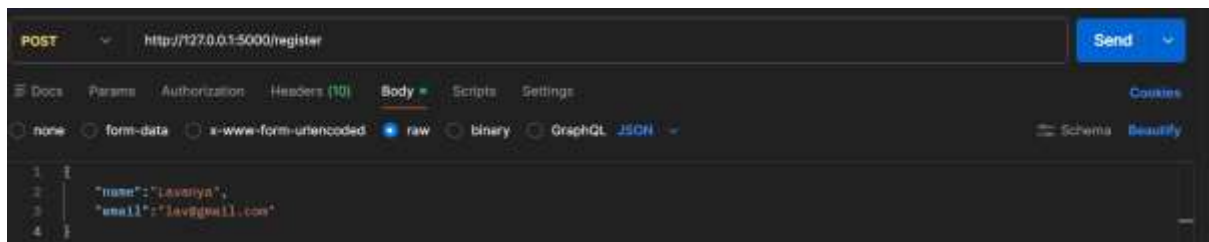
Test the API

1. Open Postman app
 - a. Request: POST <http://127.0.0.1:5000/register>
 - b. Headers: Content-Type: application/json



A. SUCCESSFUL REGISTRATION

Body: {"name":"Lavanya","email":"lav@gmail.com"}



Response

```
{
  "message": "Student registered successfully.",
  "status": "Success",
  "student": {
    "email": "lav@gmail.com",
    "name": "Lavanya"
  }
}
```

Status Code: 200 OK

B. INVALID EMAIL

Body: {"name":"Lavanya","email":"lav "}

Response:

```
{
  "message": "Invalid email format.",
  "status": "Error"
}
```

Status Code: 400 BAD REQUEST

C. MISSING NAME OR EMAIL

Body: {"name":"","email":""}

Response:

```
{  
  "message": "Name and Email are required.",  
  "status": "Error"  
}
```

Status Code: 400 BAD REQUEST

LAB PROGRAM – 03

Implement CRUD API with Database:

- a. Create a “students” table in SQLite/MySQL.
- b. Implement API to insert and fetch student records.

Code: Create a file called as lab3.py

```
from flask import Flask, jsonify, request
from flask_mysqldb import MySQL

app = Flask(__name__)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ""
app.config['MYSQL_DB'] = 'flask_crud'

mysql = MySQL(app)

# Create table
'''cur = mysql.connection.cursor()
cur.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        email VARCHAR(100) NOT NULL,
        phone VARCHAR(15) NOT NULL
    )
''')
mysql.connection.commit()
cur.close()'''
```

```
# ----- GET ALL STUDENTS -----
```

```
@app.route('/students', methods=['GET'])
```

```
def get_students():
```

```
    try:
```

```
        cur = mysql.connection.cursor()
```

```
        cur.execute("SELECT * FROM students")
```

```
        students = cur.fetchall()
```

```
        cur.close()
```

```
        data = []
```

```
        for student in students:
```

```
            data.append({
```

```
                "id": student[0],
```

```
                "name": student[1],
```

```
                "email": student[2],
```

```
                "phone": student[3]
```

```
            })
```

```
        return jsonify({
```

```
            "success": True,
```

```
            "message": "Students retrieved successfully.",
```

```
            "count": len(data),
```

```
            "data": data
```

```
        }), 200
```

```
    except Exception as e:
```

```
        return jsonify({
```

```
            "status": "error",
```

```
            "message": str(e)
```

```
        }), 500
```

```
# ----- INSERT STUDENT -----
```

```
@app.route('/students', methods=['POST'])
```

```
def insert_student():
```

```
    try:
```

```
        data = request.get_json()
```

```

if not data:
    return jsonify({
        "success": False,
        "message": "Request body is required."
    }), 400
name = data.get("name")
email = data.get("email")
phone = data.get("phone")

if not name or not email or not phone:
    return jsonify({
        "success": False,
        "message": "Name, Email and Phone are required."
    }), 400
cur = mysql.connection.cursor()
cur.execute(
    "INSERT INTO students(name,email,phone) VALUES(%s,%s,%s)",
    (name, email, phone)
)
mysql.connection.commit()
student_id = cur.lastrowid
cur.execute("SELECT * FROM students WHERE id=%s", (student_id,))
student = cur.fetchone()
cur.close()
return jsonify({
    "success": True,
    "message": "Student added successfully.",
    "data": {
        "id": student[0],
        "name": student[1],
        "email": student[2],
        "phone": student[3]
    }
})

```

```

    }
    }), 201
except Exception as e:
    mysql.connection.rollback()
    print(str(e))
    return jsonify({
        "success": False,
        "message": str(e)
    }), 500

# ----- RUN -----
if __name__ == '__main__':
    app.run(debug=True)

```

LAB PROGRAM – 04

Extend CRUD with Update and Delete:

- a. Add API endpoints to update student name by ID.
- b. Add endpoint to delete a student record.

```

from flask import Flask, jsonify, request
from flask_mysql import MySQL

app = Flask(__name__)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ""
app.config['MYSQL_DB'] = 'flask_crud'

mysql = MySQL(app)

# ----- UPDATE STUDENT -----

```

```

@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    try:
        data = request.get_json()

        if not data:
            return jsonify({
                "success": False,
                "message": "Request body is required."
            }), 400

        name = data.get("name")
        email = data.get("email")
        phone = data.get("phone")

        if not name or not email or not phone:
            return jsonify({
                "success": False,
                "message": "Name, Email and Phone are required."
            }), 400

        cur = mysql.connection.cursor()

        cur.execute("SELECT * FROM students WHERE id=%s", (id,))
        student = cur.fetchone()

        if student is None:
            cur.close()
            return jsonify({
                "success": False,
                "message": "Student not found."
            }), 404

```

```

cur.execute("""
    UPDATE students
    SET name=%s, email=%s, phone=%s
    WHERE id=%s
    """, (name, email, phone, id))

mysql.connection.commit()

cur.execute("SELECT * FROM students WHERE id=%s", (id,))
student = cur.fetchone()
cur.close()

return jsonify({
    "success": True,
    "message": "Student updated successfully.",
    "data": {
        "id": student[0],
        "name": student[1],
        "email": student[2],
        "phone": student[3]
    }
}), 200

except Exception as e:
    mysql.connection.rollback()
    return jsonify({
        "success": False,
        "message": str(e)
    }), 500

# ----- DELETE STUDENT -----
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):

```

```

try:
    cur = mysql.connection.cursor()

    cur.execute("SELECT * FROM students WHERE id=%s", (id,))
    student = cur.fetchone()

    if student is None:
        cur.close()
        return jsonify({
            "success": False,
            "message": "Student not found."
        }), 404

    cur.execute("DELETE FROM students WHERE id=%s", (id,))
    mysql.connection.commit()
    cur.close()

    return jsonify({
        "success": True,
        "message": "Student deleted successfully."
    }), 200

except Exception as e:
    mysql.connection.rollback()
    return jsonify({
        "success": False,
        "message": str(e)
    }), 500

# ----- RUN -----
if __name__ == '__main__':
    app.run(debug=True)

```

